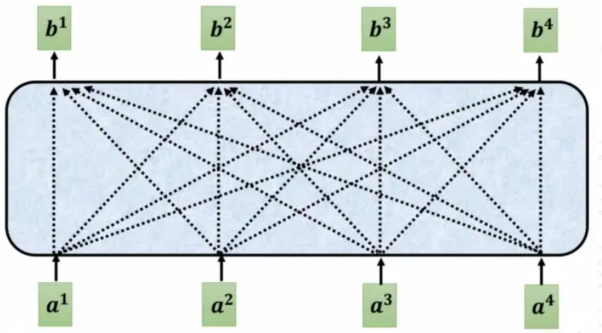
1 (2pt). Self-Attention by hand

We have a sentence of four words. The corresponding word embeddings a^1, a^2, a^3 and a^4 are shown below.

$$a_1 = [1, 1, 1]^T$$
$$a_2 = [2, 2, 2]^T$$
$$a_3 = [3, 3, 3]^T$$
$$a_4 = [4, 4, 4]^T$$

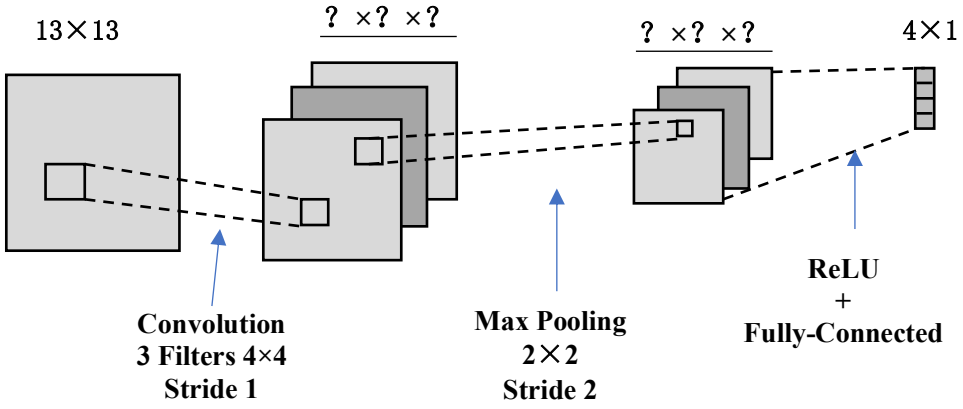
Now we are using one simple self-attention layer to process these inputs.



Please compute the new hidden state b^1 for the first word. (Suppose that all the model parameters you'll need are initialized to ones)

2 (2pt). Convolutional Neural Networks

Below is a diagram of a small convolutional neural network that converts a 13x13 image into 4 output values. The network has the following layers/operations from input to output: convolution with 3 filters, max pooling, ReLU, and finally a fully-connected layer. For this network we will not be using any bias/offset parameters (w_0). Please answer the following questions about this network.



a) (0.5pt) What is the size of the image/feature map after each layer?

b) (0.5pt) How many weights in the convolutional layer do we need to learn?

c) (0.5pt) How many ReLU operations are performed on the forward pass?

d) (0.5pt) How many weights do we need to learn for the entire network?

3 (1pt). LLM Sampling

Assume that we have run the LLM to get the logits of the next token

token	dog	cat	you	the	a	hello	fish	apple	banana	egg
logits	2.1	1.9	3.3	-1.2	0.0	-2.9	1.9	-0.2	0.8	1.1

Please implement the effects of different sampling parameters used in LLMs, including **temperature**, **top_k**, and **top_p**.

Temperature is used to control the randomness of the model's predictions by adjusting the softmax distribution. Given logits$x_1, x_2, \ldots, x_n$, the temperature-adjusted softmax is:

$$softmax_with_temperature(x, T) = \frac{e^{x_j/T}}{\sum_{j=1}^{n} e^{x_j/T}}$$

Top-k sampling limits choices to the k most probable tokens and removes the tail of the distribution, which can prevent sampling low-quality or irrelevant words.

1. Sort tokens by probability in descending order.
2. Keep the top k tokens.
3. Re-normalize their probabilities using softmax

Top-p selects the **smallest set of tokens** whose cumulative probability exceeds a threshold.

1. Sort tokens by descending probability.
2. Select the minimal set S such that: $\sum_{i \epsilon S} p_i \geq topp$
3. Re-normalize their probabilities using softmax

a) (0.25pt) Use softmax to compute probabilities from the logits. Sort the tokens in descending order of probability and list the corresponding tokens and their probabilities.

b) (0.25pt) Compute probabilities from the logits using temperature = 2.0. Sort the tokens in descending order of probability and list the corresponding tokens and their probabilities.

c) (0.25pt) From the results in b), apply top_k = 5. Sort the remaining tokens in descending order of probability and list the corresponding tokens and their probabilities.

d) (0.25pt) From the results in step c), apply top_p = 0.5. Sort the remaining tokens in descending order of probability and list the corresponding tokens and their probabilities.

Hint: You can compute results using Python numpy. For your convenience, we provide the data here.

```
logits = {
   'dog': 2.1, 'cat': 1.9, 'you': 3.3, 'the': -1.2, 'a': 0.0,
   'hello': -2.9, 'fish': 1.9, 'apple': -0.2, 'banana': 0.8, 'egg': 1.1
}
```